

并行计算笔记整理(1)(gpt5.5 plus)

主讲：陈国良教授 原课程链接：[超算学堂EasyHPC 并行计算 - 结构·算法·编程](#)

主要内容

Part1

1. 并行计算机系统结构分类
2. MPP、SMP、COW、DSM 对比
3. 并行计算机存储访问模型
4. 网络环境分类
5. 互连拓扑
6. 静态连接网络
7. 动态连接网络
8. 宽带网络技术
9. 系统加速比定律
10. 可扩展性 Scalability
11. Iso-Efficiency, 等效率函数
12. Iso-Speed, 等速度指标
13. Average Latency, 平均延迟
14. 本章总复习

Part2

1. PRAM Model
2. APRAM Model
3. BSP Model
4. More About BSP
5. LogP Model
6. BSP vs LogP
7. C³ Model
8. 真实并行计算模型对比
9. 并行算法设计方法
10. 并行算法设计技巧
11. PCAM Design Methodology
12. PCAM: Partitioning
13. PCAM: Communication
14. PCAM: Agglomeration
- Surface-to-Volume Effects
15. PCAM: Mapping
16. Switching Techniques
17. X-Y Routing Algorithm
18. E-Cube Routing Algorithm
19. Cannon's Algorithm
20. DNS Algorithm
21. Gauss-Seidel Algorithm
22. 本章核心总结
23. 复习重点

主要内容

- Part I: the contemporary parallel computer system architectures and memory access models, parallel system interconnections and parallel system performance evaluation.

Hardware Platform for Parallel Computing --- Parallel Computer Systems

- System Architectures and Models
 - System Interconnections
 - Performance Evaluation
- Part II: the parallel computational models, the design methods, techniques and methodology of parallel algorithms, as well as some parallel numerical algorithms.

Theoretical Base for Parallel Computing --- Parallel Algorithms

- Computational Models
 - Design Methods
 - Design Techniques
 - Design Methodology
 - Parallel Numerical Algorithms
- Part III: the parallel programming models, shared-memory, message-passing and data parallel programming, as well as parallel programming environment and tools.

Software Support for Parallel Computing --- Parallel Programming

- Programming Models
 - Shared-Memory Programming
 - Message-Passing Programming
 - Data-Parallel Programming
 - Programming Environment and Tools

Part1

下面按课件顺序，把这一章 **Part I: Parallel Computer System Architectures** 的内容完整讲一遍。这份课件主要讲并行计算机体系结构、存储访问模型、互连网络、宽带网络、加速比定律和可扩展性评价方法。内容来自你上传的 `partI.pdf`。

1. 并行计算机系统结构分类

这一部分先介绍几类典型并行计算机：**PVP、SMP、MPP、DSM、COW/Cluster**。

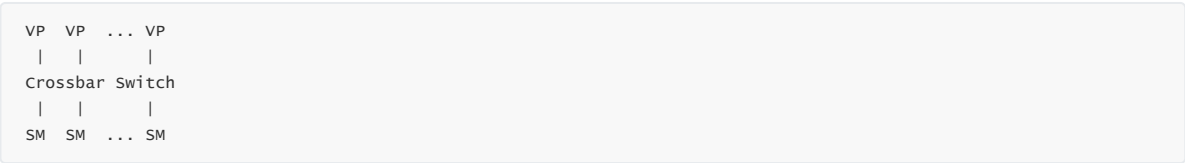
1.1 PVP: Parallel Vector Processors，并行向量处理机

PVP 是早期高性能计算机的重要形态。

它的特点是：

- 属于 **MIMD**，即多指令流、多数据流。
- 使用 **UMA**，所有处理器访问共享内存的时间基本一致。
- 并行粒度较大，适合大规模科学计算中的向量运算。
- 处理器数量不多，但每个处理器都非常强，是专门设计的向量处理器。
- 通过高带宽交叉开关连接向量处理器和共享内存模块。
- 通常有大量向量寄存器和指令缓冲区，但一般没有普通意义上的 cache。
- 例子包括 Cray C-90/T-90、NEC SX-4、Galaxy-1。

课件图中结构是：



其中 VP 是 Vector Processor，SM 是 Shared Memory。
PVP 的优势是单处理器向量性能极强，适合规则、密集数值计算；缺点是硬件专用、成本高、扩展性有限。

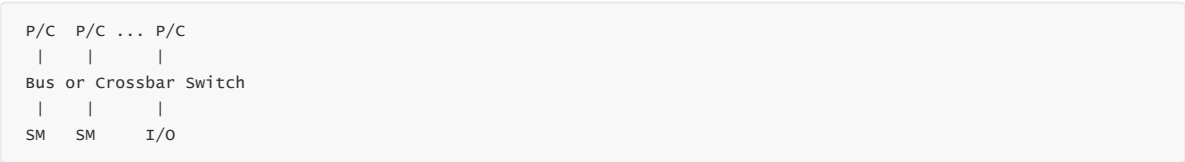
1.2 SMP: Symmetric Multiprocessors，对称多处理机

SMP 是共享内存多处理器系统。

它的特点是：

- 属于 **MIMD + UMA**。
- 并行粒度中等。
- 使用商品化微处理器，每个处理器通常带片上或片外 cache。
- 处理器之间通过高速总线或交叉开关连接。
- 所有处理器共享中央内存。
- “对称”指每个处理器对共享内存、I/O 和操作系统服务的访问地位相同。
- 例子包括 SGI Power Challenge、DEC Alpha Server 8400、Dawning-1。

SMP 的典型结构是：



P/C 表示 Processor/Cache。

SMP 的核心优点是编程容易，因为它提供共享内存模型。程序员可以像写多线程程序一样共享变量。
但它的主要问题是 **不可无限扩展**：共享内存和总线会成为瓶颈。处理器越多，总线竞争和 cache 一致性维护开销越大。

课件第 3 页比较了五种商业 SMP 系统，包括 DEC AlphaServer、HP 9000/T600、IBM RS6000/R40、Sun Ultra Enterprise 6000、SGI Power Challenge XL，比较指标包括处理器数量、处理器类型、cache、最大内存、互连带宽、磁盘、I/O 通道、I/O 插槽和 I/O 带宽。

1.3 MPP: Massively Parallel Processors, 大规模并行处理机

MPP 是由大量处理节点组成的并行机。

特点:

- 属于 **MIMD + NUMA**。
- 并行粒度中等或较大。
- 使用大量商品化微处理器。
- 通过专门设计的高带宽、低延迟通信网络连接。
- 内存在物理上是分布式的, 可以是共享的, 也可以是不共享的。
- 节点可能有本地磁盘, 也可能没有。
- 通常通过阻塞式消息传递进行同步。
- 例子包括 Intel Paragon、IBM SP2、Dawning-1000。

典型结构:

Node = P/C + LM + NIC
多个 Node 通过 Custom-designed Network 互连

其中:

- P/C: 处理器/cache
- LM: Local Memory, 本地内存
- NIC: Network Interface Card, 网络接口
- MB: Memory Bus 或节点内部总线

MPP 与 SMP 最大区别在于:

SMP 是共享内存, MPP 更强调分布式内存和消息传递。

MPP 可扩展性强, 但编程复杂度更高。

课件第 5 页比较了三个 MPP 系统: Intel/Sandia ASCI Option Red、IBM SP2、SGI/Cray Origin 2000。比较内容包括规模、可用时间、处理器类型、节点结构、互连和内存模型、操作系统、原生编程机制以及其他编程模型。

1.4 ASCI Option-Red System

第 6 页展示了 ASCI Option-Red 系统结构。

这个系统由多类节点构成:

- Computation Nodes: 计算节点
- Service Nodes: 服务节点
- I/O Nodes: 输入输出节点
- System Nodes: 系统节点
- Boot Node: 启动节点
- Node Station: 节点管理站
- 外部设备: 磁盘、磁带、Boot RAID 等

它通过 Ethernet、ATM、HiPPI 等网络连接不同部分。

这张图想说明: 大型 MPP 系统不是单纯堆很多计算节点, 还需要 I/O、服务、启动、管理、存储等辅助节点共同构成完整系统。

1.5 MPP 使用的高性能 CPU 芯片

第 7、8 页介绍当时用于 MPP 的高性能 CPU 家族。

课件列举了:

- Intel x86 系列: 8086、286、386、486、Pentium、Pentium Pro
- Motorola 系列: M68x0、680x0
- Digital Alpha: 21064、21164、21264
- MIPS 系列: R2000、R3000、R4000、R5000、R8000、R10000
- HP PA-RISC: PA 7300、PA 8000
- Sun SPARC: SPARC、MicroSPARC、SuperSPARC、UltraSPARC
- PowerPC: 601、603、604e、620、630
- 嵌入式 RISC、DSP、微控制器、媒体处理器等

这一部分的目的是不是要求背芯片参数, 而是说明: MPP 常采用商品化高性能微处理器作为节点核心。

1.6 DSM: Distributed Shared-Memory, 分布式共享内存

DSM 是介于共享内存和分布式内存之间的一种模型。

特点:

- 属于 **MIMD**。
- 存储访问上涉及 **NUMA** 和 **NORMA** 的特点。
- 并行粒度较大。
- 内存在物理上分布于各节点。
- 但系统硬件和软件向应用程序提供统一地址空间。
- 使用目录机制 DIR, 即 cache directory, 维护分布式 cache 的一致性。
- 使用专门设计的通信网络。
- 支持共享内存编程风格。
- 例子包括 Stanford DASH、Cray T3D。

典型结构:

Node = P/C + LM + DIR + NIC

多个 Node 通过 Custom-designed Network 互连

DSM 的关键思想是:

物理上分布, 逻辑上共享。

程序员看到的是共享地址空间, 但底层实际可能跨节点访问远程内存。
因此 DSM 既想保留共享内存编程的便利, 又想获得分布式系统的可扩展性。

1.7 COW: Cluster of Workstations, 工作站集群

COW 是由多台完整工作站或 PC 组成的集群系统。

特点:

- 属于 **MIMD + NUMA**。
- 并行粒度较粗。
- 分布式内存。
- 每个节点本身就是一台完整计算机, 可以是 SMP 或 PC。
- 每个节点通常有完整操作系统。
- 使用低成本商品网络, 如 Ethernet、ATM。
- 每个节点通常有本地磁盘。
- 与 MPP 不同, MPP 节点通常只运行微内核, 而 COW 节点运行完整 OS。
- 例子包括 Berkeley NOW、Alpha Farm、FXCOW。

典型结构:

Node = P/C + Memory + Bridge + Local Disk + I/O Bus + NIC

多个节点通过 Commercial Networks 互连

COW 的优势是成本低、易构建、可利用已有工作站; 缺点是网络延迟较高、系统管理和通信效率不如专用 MPP。

第 11 页列举了多个 COW 研究项目, 如 Berkeley NOW、Princeton SHRIMP、Wisconsin Wind Tunnel、Syracuse WWVM、Virtual Legion 等, 主要方向包括网络通信、共享虚拟内存、分布式处理、元计算等。

2. MPP、SMP、COW、DSM 对比

第 12、13 页从多个维度比较这些系统。

可以归纳为:

类型	存储方式	编程风格	典型通信	扩展性	成本
SMP	集中共享内存	共享变量/线程	总线或交叉开关	较差	中高
MPP	分布式内存	消息传递	专用网络	很好	高
COW/Cluster	分布式内存	消息传递	商品网络	较好	低
DSM	物理分布、逻辑共享	共享内存	目录一致性 + 网络	较好	较高

几个关键指标:

- **节点数量**: MPP 和 DSM 可达到上千节点, SMP 通常十几个或几十个, Cluster 也可达到较大规模。

- **通信机制**：SMP 主要共享内存，MPP 多用消息传递，DSM 用共享内存模型加一致性协议，COW 多用消息传递。
- **操作系统**：SMP 往往是单一 OS，MPP 节点上可能是微内核，Cluster 是每个节点完整 OS，DSM 可能有多个 OS。
- **地址空间**：SMP 单一地址空间，MPP 多地址空间，Cluster 多地址空间，DSM 逻辑上共享地址空间。
- **网络协议**：Cluster 常用标准网络协议，MPP/DSM 常用非标准或专用网络。
- **系统可用性**：MPP 和 DSM 中等，SMP 可能较低，Cluster 可通过冗余提高可用性。
- **性能指标**：MPP 常看吞吐量和周转时间，SMP 常看周转时间，Cluster 常看吞吐量，DSM 常看响应时间。

3. 并行计算机存储访问模型

这一部分解释 UMA、NUMA、COMA、NORMA 以及它们和系统结构之间的关系。

3.1 UMA：Uniform Memory Access，统一内存访问

UMA 的核心是：

所有处理器访问所有内存字的时间相同。

特点：

- 物理内存由所有处理器统一共享。
- 所有处理器对内存访问时间相等。
- 适合通用计算和分时系统。
- 常见于 SMP。

典型结构：

```
Processors
|
System Interconnect
|
I/O + Shared Memory Modules
```

优点是简单、易编程。

缺点是随着处理器数量增加，共享总线或共享内存会成为瓶颈。

3.2 NUMA：Non-Uniform Memory Access，非统一内存访问

NUMA 的核心是：

内存逻辑上共享，但访问不同位置内存的时间不同。

特点：

- 共享内存由各处理器的本地内存 LM 组成。
- 访问本地内存较快。
- 访问其他处理器的远程内存较慢。
- 通常通过互连网络连接各节点。

可以理解为：

NUMA 是为了提高扩展性，把内存分布到各节点，但仍维持某种共享内存视图。

3.3 CC-NUMA：Cache-Coherent NUMA

第 16 页介绍 CC-NUMA 系统。

CC-NUMA 的意思是：

带 cache 一致性的 NUMA 系统。

每个节点里有：

- 多个处理器/cache
- 本地内存
- I/O
- NIC：网络接口
- DIR：cache directory，目录
- RC：remote cache，远程 cache

节点之间通过系统互连网络连接。

CC-NUMA 的难点是 cache 一致性。

因为每个节点都有 cache，如果多个处理器缓存了同一个内存块，就必须保证读写结果一致。目录 DIR 就是用来记录数据副本位置和状态的。

3.4 COMA: Cache-Only Memory Access

COMA 是 NUMA 的特殊情况。

特点:

- 分布式内存被转换成 cache。
- 所有 cache 共同形成全局地址空间。
- 远程 cache 访问由分布式 cache 目录辅助。
- 没有固定的“主存位置”，数据可以像 cache 块一样迁移。

COMA 的思想是：

既然 NUMA 中远程访问慢，那就尽量让数据迁移到需要它的节点附近，从而提高局部性。

但 COMA 系统实现复杂，因此实际应用不如 NUMA/CC-NUMA 常见。

3.5 NORMA: No-Remote Memory Access

NORMA 的核心是：

所有内存都是私有的，处理器不能直接访问远程内存。

特点:

- 每个节点只能直接访问自己的本地内存。
- 节点之间通过消息传递通信。
- 属于分布式内存多计算机系统。
- 常见于 MPP 和 Cluster。

如果一个处理器需要另一个节点的数据，不能直接 load/store，只能发送消息请求。

这就是 MPI 程序的典型模型：

```
Process A --send/receive--> Process B
```

NORMA 的优势是结构清晰、扩展性好；缺点是编程复杂，需要显式通信。

3.6 体系结构与存储访问模型的关系

第 19 页把这些模型串起来：

```
MIMD
├── Multiprocessors: 单地址空间，共享内存
│   ├── UMA: PVP、SMP
│   └── NUMA
│       ├── COMA
│       ├── CC-NUMA
│       └── NCC-NUMA
└── Multicomputers: 多地址空间，非共享内存
    ├── NORMA: Cluster、MPP
    └── DSM: 通过软件/硬件模拟共享内存
```

也可以这样记：

- **SMP/PVP**：集中共享内存，UMA。
- **CC-NUMA/COMA**：分布式共享内存，NUMA。
- **MPP/Cluster**：分布式私有内存，NORMA。
- **DSM**：物理分布，但向用户提供共享内存视图。

4. 网络环境分类

第 20 页介绍五种网络环境。

4.1 Bus，总线

包括：

- Processor bus
- Memory bus
- I/O bus

通常位于主板或背板上，距离很短，一般不超过 2 米。

主要用于单机内部部件连接。

4.2 SAN：System Area Network，系统区域网络

SAN 用来把多个节点连接成一个单一系统。
距离通常是 3–25 米。
常用于机柜内或机房内的高性能集群互连。

4.3 LAN：Local Area Network，局域网

LAN 覆盖一个建筑、校园或企业，距离约 25–500 米。
用于连接多个系统。
典型例子是 Ethernet、FDDI。

4.4 MAN：Metropolitan Area Network，城域网

MAN 覆盖一个城区或城市范围，一般可到 25 km 左右。

4.5 WAN：Wide Area Network，广域网

WAN 是多个较小网络组成的互联网络，可用于元计算、网格计算等跨地域计算。

5. 互连拓扑

互连网络分为两类：

- **静态连接网络**：连接关系固定。
- **动态连接网络**：连接关系可根据通信需求动态改变。

6. 静态连接网络

第 21、22 页介绍静态拓扑，包括：

- Linear array：线性阵列
- Ring：环
- Mesh：网格
- Tree：树
- Hypercube：超立方体
- CCC：Cube-Connected Cycles
- Torus：环绕网格
- Completely connected：全连接
- Star：星型

比较指标包括：

- **节点度 d**：每个节点连接多少条边。
- **网络直径 D**：任意两节点之间最短路径的最大值。
- **链路数 I**：总连接数。
- **二分带宽 B**：把网络切成两半时跨越切口的最小链路数。
- **对称性**：从任意节点看网络结构是否一致。
- **规模与备注**。

几个典型拓扑：

6.1 线性阵列

P1 - P2 - P3 - ... - Pn

- 节点度最大为 2。
 - 直径为 N-1。
 - 结构简单，但通信距离可能很长。
 - 扩展容易，但性能较差。
-

6.2 环

$P_1 - P_2 - \dots - P_n - P_1$

- 节点度为 2。
- 直径约为 $N/2$ 。
- 比线性阵列更好，因为首尾相连。
- 对称性好。

6.3 全连接

每个节点都直接连到其他所有节点。

- 直径为 1。
- 通信最快。
- 但链路数是 $N(N-1)/2$ ，硬件成本极高。
- 不适合大规模系统。

6.4 二叉树

- 适合层次化通信。
- 节点度通常为 3。
- 直径与树高相关。
- 根节点附近容易成为瓶颈。
- 对称性较差。

6.5 2D Mesh，二维网格

节点按二维矩阵排列，每个节点连接上下左右。

- 节点度通常为 4。
- 结构规则，易实现。
- 适合二维科学计算问题。
- 边缘节点度较低，不完全对称。

6.6 2D Torus，二维环面

Torus 是 Mesh 的改进：左右边界相连，上下边界相连。

- 节点度为 4。
- 比 Mesh 直径更小。
- 对称性更好。
- 常用于大规模并行机网络。

6.7 Hypercube，超立方体

n 维超立方体有 $N = 2^n$ 个节点。

- 节点度为 n 。
- 直径为 n 。
- 对称性好。
- 通信性能较好。
- 但节点度随规模增长，硬件连接复杂度增加。

6.8 CCC：Cube-Connected Cycles

CCC 是超立方体的一种变体，用环结构降低节点度。

- 节点度固定为 3。
- 相比超立方体硬件连接更简单。
- 但通信距离变长。

7. 动态连接网络

动态连接网络包括总线、交叉开关和多级互连网络。

7.1 Bus，总线

总线用于 master 设备和 slave 设备之间传输数据，例如处理器和内存之间。

课件区分：

- **System bus / I/O bus**：位于背板或中心板上，用于 CPU 板、内存板、I/O 板等连接。
- **Local bus**：局部总线，例如内存板上的 memory bus。

总线结构简单、成本低，但同一时间通常只能有有限设备通信，因此扩展性差。

7.2 Crossbar Switch，交叉开关

交叉开关是单级交换网络。

特点：

- 有 n 个端口时，复杂度为 n^2 。
- 每个输入和输出之间有交叉点。
- 交叉点可以动态打开或关闭。
- 可以同时支持多个互不冲突的通信对。
- 可用于处理器之间通信，也可用于处理器访问内存。

优点是带宽高、冲突少；缺点是硬件复杂度随 n^2 增长，不适合无限扩展。

7.3 MIN：Multistage Interconnection Networks，多级互连网络

MIN 用多个交换级组成互连网络。

特点：

- 延迟数量级是 $O(\log n)$ 。
- 构造模块数量是 $O(n \log n)$ 。
- 可实现输入到输出的部分排列。
- 成本介于总线和交叉开关之间。
- 常用于大规模并行机互连。

可以理解为：

MIN 想在交叉开关的高性能和总线的低成本之间折中。

7.4 动态网络比较

第 26 页比较了 System Bus、Multistage Network、Crossbar Switch。

参数包括：

- n ：连接端点数量
- w ：数据通道宽度
- k ：交换模块输入数
- f ：时钟频率

总体结论：

- **总线**：硬件复杂度低，但带宽有限。
 - **多级网络**：复杂度和性能折中，适合较大规模。
 - **交叉开关**：性能高，但复杂度最高。
-

8. 宽带网络技术

这一部分介绍 FDDI、Ethernet、Myrinet、ATM、SCI，并进行比较。

8.1 FDDI: Fiber Distributed Data Interface

FDDI 是光纤分布式数据接口。

特点:

- 使用两个方向相反的环。
- 双环提供冗余路径，提高可靠性。
- 传输速率 100–200 Mbps。
- 距离：
 - 铜线约 100 m
 - 多模光纤约 2 km
 - 单模光纤约 60 km
- 缺点是不适合支持多媒体流量。

课件图展示了 Dual FDDI rings 作为骨干网络，连接文件服务器、数据库服务器、计算服务器、路由器、以太网 hub 和桌面机。

8.2 Fast Ethernet 与 Gigabit Ethernet

课件比较三代 Ethernet:

代际	带宽	大致引入年份	主要应用
Ethernet 10BaseT	10 Mbps	1982	文件共享、打印共享
Fast Ethernet 100BaseT	100 Mbps	1994	工作组、服务器、客户端/服务器
Gigabit Ethernet	1 Gbps	1997	大图像、多媒体、Intranet、Internet、数据仓库

课件还展示了 Gigabit Ethernet LAN backbone 的结构:

核心是 1Gb/s Ethernet Switch，向下连接 100Mb/s switch，再连接 10Mb/s 或 100Mb/s 终端。

这说明 Ethernet 从普通局域网逐渐发展为可用于集群和服务器骨干的高速网络。

8.3 Myrinet SAN & LAN

Myrinet 是一种面向集群的高速网络。

目标:

- 用商品化产品构建计算机集群。
- 支持机柜内 SAN 集群。
- 支持基于 LAN 的桌面主机和服务器集群。

特点:

- 定义在数据链路层。
- 使用 wormhole routing，即虫孔路由。
- 使用 crossbar switch 转发包。
- 采用无死锁路由机制。
- 多个数据包可以同时通过 Myrinet。
- 提供 Myrinet/SBUS 给 Sun 工作站，Myrinet/PCI 给 PC。
- 光纤接口也在发展中。
- 局限是主机接口依赖总线类型，限制了对多种主机的兼容。

Myrinet 在高性能集群中强调低延迟和高吞吐。

8.4 ATM: Asynchronous Transfer Mode, 异步传输模式

ATM 是一种与介质无关的信息传输协议。

特点:

- 将流量切分成固定长度的 53 字节 cell。
- 适用于语音、视频、数据统一传输的设想。
- 通过 ATM 网络交换 cell，最后再重组为 packet。

缺点:

- 缺乏统一接受的 cell 管理和网络协议。
- 电话行业和计算机行业标准不一致。
- 如果标准不能统一，ATM 的缺点可能超过其优势。

ATM 架构中涉及多个层次，包括物理层、ATM 层、AAL 层等，用来完成信元生成、复用、拆装、重组等功能。

8.5 SCI: Scalable Coherence Interface

SCI 的动机是:

前面介绍的许多网络都连接到节点的 I/O 总线, 节点间通信依赖消息传递。
这种 I/O 通信比 SMP 中共享内存通信慢。

SCI 的目标是把传统背板总线扩展为:

- 全双工
- 点到点
- 低延迟
- 高带宽
- 支持 DSM coherent cache image 的互连结构

课件中给出性能目标:

- 延迟 < 1 ms
- 带宽 8 GB/s

SCI 可以理解作为一种试图把共享内存一致性机制扩展到更大系统的互连技术。

8.6 网络技术比较

第 32 页比较了:

- Gigabit Ethernet
- HiPPI
- Fiber Channel
- FDDI
- SCI
- ATM
- Myrinet

比较维度包括:

- 架构: 共享介质、交换式、混合式
- 介质类型: UTP、同轴、光纤、STP、电链路等
- 标准组织: IEEE、ANSI、ATM Forum、IETF、ITU-TSS、Myricom 等

总体理解:

- Ethernet 标准化程度高、成本低、生态强。
 - Myrinet 面向高性能集群, 低延迟。
 - SCI 强调一致性和共享内存扩展。
 - ATM 强调统一传输但标准复杂。
 - FDDI 曾用于高速骨干, 但后来被 Ethernet 取代趋势明显。
-

9. 系统加速比定律

这一部分讲三个经典性能模型:

- Amdahl's Law
- Gustafson's Law
- Sun and Ni's Law

它们都讨论并行系统能获得多大加速。

设:

- W: 总工作量
 - Ws: 串行部分工作量
 - Wp: 可并行部分工作量
 - p: 处理器数量
 - $f = W_s / W$: 串行比例
-

9.1 Amdahl's Law: 固定问题规模

Amdahl 定律适用于:

 问题规模固定, 处理器增加后, 运行时间减少。

总工作量:

$$W = W_s + W_p$$

并行执行时间：

$$T_p = W_s + W_p / p$$

加速比：

$$S = W / (W_s + W_p / p)$$

如果 $f = W_s / W$ ，则：

$$S = p / (1 + f(p - 1))$$

当 $p \rightarrow \infty$ 时：

$$S \rightarrow 1 / f$$

这说明：

只要程序中存在串行部分，加速比就有上限。

例如课件中 $p = 1024$ ：

$$S_{1024} = 1024 / (1 + 1023f)$$

如果串行比例：

- $f = 0\%$ ，理论加速比 1024
- $f = 1\%$ ，加速比约 91
- $f = 2\%$ ，约 48
- $f = 3\%$ ，约 31
- $f = 4\%$ ，约 24
- $f = 100\%$ ，加速比 1

结论：

Amdahl 定律强调串行瓶颈非常致命。

9.2 Gustafson's Law: 固定时间

Gustafson 定律适用于：

运行时间固定，但随着处理器增加，可以解决更大问题。

在大型科学计算中，增加处理器不是为了把同一个小问题算得更快，而是为了在相同时间内算更大、更精确的问题。

Gustafson 定律：

$$S' = f + p(1 - f)$$

也可写成：

$$S' = p - f(p - 1)$$

对 $p = 1024$ ：

$$S'_{1024} = 1024 - 1023f$$

如果 $f = 1\%$ ，加速比约为 1014，而不是 Amdahl 定律中的约 91。

这说明：

在固定时间、扩展问题规模的场景下，并行系统的实际收益可能远高于 Amdahl 定律预测。

9.3 Sun and Ni's Law: Memory Bounding, 受内存约束的扩展定律

Sun 和 Ni 定律进一步推广 Amdahl 和 Gustafson。

核心思想：

随着处理器数量增加，内存容量也增加，因此可以解决受内存容量限制的最大问题。

假设并行部分工作量可以放大 $G(p)$ 倍， $G(p)$ 表示由于内存容量随 p 增加而带来的工作量增长。

加速比：

$$S = [Ws + G(p)wp] / [Ws + G(p)wp / p]$$

若 $f = Ws / W$ ，则：

$$S = [f + G(p)(1 - f)] / [f + G(p)(1 - f) / p]$$

这个模型说明：

- Amdahl：问题规模固定。
- Gustafson：时间固定，问题规模随处理器增加。
- Sun-Ni：问题规模受内存容量限制，内存随处理器增加而增加。

Sun-Ni 更适合解释大规模科学计算，因为许多问题不是 CPU 不够，而是内存容量限制了可处理问题规模。

10. 可扩展性 Scalability

第 36 页开始讲系统可扩展性。

10.1 定义

一个计算机系统，包括硬件、软件、算法，如果能够：

- 增加资源以满足性能和功能需求；
- 或减少资源以降低成本；

就称为 scalable，可扩展。

10.2 可扩展性的三个维度

机器规模可扩展性

关注增加处理器后性能是否提升。

也衡量系统最多能支持多少处理器。

例如：

SMP 由于共享总线和共享内存瓶颈，机器规模可扩展性较差。

MPP/Cluster 通常更好。

问题规模可扩展性

关注系统能否处理更大数据规模和更大工作量。

例如：

一个系统在 100 个处理器上跑小问题效率很低，但在大问题上效率很好，这说明它的问题规模可扩展性较好。

技术可扩展性

关注当底层技术变化时，系统性能是否能继续提升。

例如：

CPU 更快、内存更大、网络更快之后，体系结构是否仍能受益。

10.3 可扩展性度量方法

课件列出四种：

- Iso-efficiency
- Iso-speed
- Average latency
- Time-scale

后面重点讲前三种。

11. Iso-Efficiency，等效率函数

Iso-efficiency 关注：

当处理器数量增加时，问题规模 W 需要怎样增长，才能保持固定效率 E 。

设：

- $T1$ ：单处理器顺序时间，也就是工作量 W
- Tp ： p 个处理器并行时间
- To ：总开销
- $pTp = T1 + To$
- 加速比 $S = T1 / Tp$
- 效率 $E = S / p$

可推出：

$$S = p / (1 + To / T1) \\ E = 1 / (1 + To / T1)$$

如果固定 E ，解出 W 关于 p 的函数：

$$W = fE(p)$$

这个函数就叫 iso-efficiency function。

含义：

- 如果为了保持效率， W 只需缓慢增长，说明系统可扩展性好。
- 如果 W 必须增长很快，说明系统可扩展性差。

课件图中：

- 接近线性增长的是较好或理想可扩展性。
- 增长很陡的是 poor scalability。

优点：

Iso-efficiency 是一种分析方法，可以预测机器规模增长时所需的问题规模增长率。

缺点：

对共享内存系统来说，计算 iso-efficiency 函数并不容易，因为共享内存开销复杂。

12. Iso-Speed，等速度指标

Iso-speed 与 iso-efficiency 思想类似，但它保持的是平均速度不变，而不是效率不变。

定义平均速度：

$$V = W / (pTp)$$

当机器规模从 p 增加到 p' ，问题规模从 W 增加到 W' ，若保持平均速度不变，则定义：

$$\gamma(p, p') = p'W / (pW') = Tp / Tp'$$

当 $p = 1$ 时：

$$\gamma(1, p') = T1 / Tp'$$

含义：

Iso-speed 关注单位处理器平均完成工作的速度。

如果扩展机器后仍能保持平均速度，说明系统可扩展性好。

优点：

- speed 是更接近物理意义的指标。
- 通常可以用浮点运算数衡量。

缺点：

- 有些非浮点操作也会极大影响性能，例如通信、同步、访存、I/O。
- 只看浮点速度可能低估这些因素。

13. Average Latency，平均延迟

Average latency 是一种实验性指标，用于更底层地评价系统性能。

课件定义：

平均延迟 $L(W, p)$ 是每个处理器完成分配工作所需的平均额外开销时间。

图中展示了多个处理器 P1 到 P8，每个处理器有：

- 实际计算时间 T_i
- 启动/停止前后的 idle time
- overheads
- latency L_i
- 整体并行时间 T_{para}

当机器规模从 p 变为 p' ，并保持效率 E 不变，平均延迟指标定义为：

$$F(E, p, p') = L_e(W, p) / L_e(W', p')$$

优点：

- 是实验性指标。
- 能更精确地评价低层系统性能。
- 特别适合分析处理器延迟、通信延迟、同步延迟等。

缺点：

- 需要专用硬件和系统软件来测量处理器延迟。

课件结论：

- Iso-efficiency 与 iso-speed 在理论上是等价的。
- Average latency 可以从 iso-speed 推导出来。

14. 本章总复习

这一章可以用一张知识图串起来：

并行计算机体系结构

- ├─ PVP: 少量强向量处理器，共享内存，UMA
- ├─ SMP: 多个对称处理器，共享内存，UMA
- ├─ MPP: 大量节点，分布式内存，消息传递
- ├─ DSM: 物理分布，逻辑共享，目录一致性
- └─ COW/Cluster: 普通工作站/PC 通过商品网络连接

存储访问模型

- ├─ UMA: 访问所有内存时间相同
- ├─ NUMA: 访问本地快，远程慢
- ├─ CC-NUMA: NUMA + cache 一致性
- ├─ COMA: 内存全部 cache 化
- └─ NORMA: 不能远程访存，只能消息传递

互连网络

- ├─ 静态拓扑
 - | ── 线性阵列
 - | ── 环
 - | ── 树
 - | ── Mesh
 - | ── Torus
 - | ── Hypercube
 - | ── CCC
- └─ 动态网络
 - ├─ Bus
 - └─ Crossbar

- └─ MIN
- 性能与可扩展性
 - └─ Amdahl1: 固定问题规模, 串行部分限制加速比
 - └─ Gustafson: 固定时间, 扩大问题规模
 - └─ Sun-Ni: 受内存容量限制的问题扩展
 - └─ Iso-efficiency: 保持效率不变
 - └─ Iso-speed: 保持平均速度不变
 - └─ Average latency: 从底层延迟评价系统

最重要的理解是：

并行计算机设计一直在三者之间权衡：性能、可扩展性、编程复杂度。

- SMP 编程简单，但扩展性差。
- MPP 扩展性强，但编程复杂。
- DSM 试图兼顾共享内存便利和分布式扩展性。
- Cluster 成本低、灵活，但通信性能不如专用系统。
- 网络拓扑和存储模型直接决定系统的性能瓶颈。
- 加速比定律告诉我们，并行程序的串行部分、问题规模和内存容量都会影响最终性能。

Part2

这一章可以分成 6 大块：

- Part II 并行计算模型
 - └─ 1. 并行计算模型
 - | └─ PRAM
 - | └─ APRAM
 - | └─ BSP
 - | └─ LogP
 - | └─ C³
 - └─ 2. 并行算法设计方法
 - | └─ 并行化已有顺序算法
 - | └─ 重新设计并行算法
 - | └─ 借用经典算法思想
 - └─ 3. 并行算法设计技巧
 - | └─ Balanced Trees
 - | └─ Doubling / Pointer Jumping
 - | └─ Partitioning
 - | └─ Divide-and-Conquer
 - | └─ Pipelining
 - └─ 4. PCAM 设计方法论
 - | └─ Partitioning
 - | └─ Communication
 - | └─ Agglomeration
 - | └─ Mapping
 - └─ 5. 路由技术
 - | └─ Store-and-Forward
 - | └─ Cut-Through
 - | └─ X-Y Routing
 - | └─ E-Cube Routing
 - └─ 6. 典型并行算法
 - └─ Cannon 矩阵乘法
 - └─ DNS 矩阵乘法
 - └─ Gauss-Seidel 并行化

1. PRAM Model

PRAM 全称是 **Parallel Random Access Machine**，并行随机访问机。

课件第 1 页说，PRAM 也叫 **SIMD-SM model**，即：

- SIMD: Single Instruction Multiple Data，单指令流多数据流；
- SM: Shared Memory，共享内存。

它的基本假设是：

多个处理器 + 一个集中共享内存 + 同步操作

所有处理器在同一时刻执行同一类操作，并且可以访问统一的共享内存空间。

PRAM 的特点：

项目	含义
存储	centralized shared memory，集中共享内存

项目	含义
地址空间	single access space, 单一地址空间
操作方式	synchronous operation, 同步操作
同步方式	implicit synchronization, 隐式同步
适用粒度	fine grain parallel computation, 细粒度并行

PRAM 的优点是非常适合做理论分析。
它隐藏了很多底层细节，比如通信延迟、同步开销、cache 一致性、网络拓扑等，所以算法设计者可以专注于：

这个问题理论上能并行到什么程度？
时间复杂度是多少？
需要多少处理器？

例如并行前缀和、并行排序、并行图算法，很多理论算法都会先在 PRAM 模型下分析。

但 PRAM 的缺点也非常明显：
它不适合真实 MIMD 计算机，因为真实机器中内存访问会竞争，通信有延迟，同步也有代价。PRAM 把这些问题都理想化了。

所以 PRAM 可以理解为：

很适合算法复杂度分析，但不够贴近真实机器。

2. APRAM Model

APRAM 是 **Asynchronous PRAM**，异步 PRAM。

它相比 PRAM 的主要变化是：
PRAM 是全局同步，而 APRAM 允许异步执行。

课件第 2 页说，APRAM 也叫 **phase PRAM model**，适用于细/中粒度并行计算。

APRAM 的基本特征：

项目	含义
存储	centralized shared memory
地址空间	single access space
操作	asynchronous operation
通信	read/write shared variable communication
同步	explicit synchronization
同步机制	barrier、lock、critical region、event 等

APRAM 的计算过程由一系列 **global phases** 组成。
每个 phase 之间用 barrier 分隔；在一个 phase 内，每个处理器异步执行本地程序、读写共享内存，最后在 barrier 处同步。

可以理解为：

phase 1: 各处理器异步计算
barrier
phase 2: 各处理器异步计算
barrier
phase 3: ...

APRAM 的优点：

- 保留了 PRAM 的简单性；
- 比 PRAM 更接近真实并行程序；
- 更适合表达 barrier、lock 等显式同步；
- 复杂度分析仍然比较方便。

缺点：

- 仍然假设集中共享内存；
- 不适合分布式内存 MIMD 系统。

所以 APRAM 是：

PRAM 的现实化版本，但还没有真正进入分布式内存模型。

3. BSP Model

BSP 全称是 **Bulk-Synchronous Parallel Model**，整体同步并行模型。

这是本章非常重要的模型之一。

BSP 面向的是：

- distributed memory multicomputer，分布式内存多计算机；
- 中/大粒度并行计算；
- 异步本地计算；
- send/receive 消息通信；
- 显式 barrier 同步。

BSP 的核心思想是 **superstep**。

一个 superstep 包含三部分：

1. Local computation: 本地计算

2. Global communication: 全局通信

3. Barrier synchronization: 屏障同步

课件第 5 页图中画得很清楚：多个 virtual processors 先各自进行 local computations，然后发生 global communications，最后统一 barrier synchronization。

BSP 用三个参数描述机器：

参数	含义
p	processors, 处理器数量
g	bandwidth parameter, 带宽相关参数
l	synchronization interval, 同步代价或同步间隔

BSP 的一个 superstep 代价模型是：

$$Cost = \max(w_i) + \max(h_i)g + l$$

其中：

- w_i ：第 i 个进程的本地计算时间；
- h_i ：第 i 个进程发送或接收的最大消息包数；
- g ：每个数据包传输的时间参数，即 $1/\text{bandwidth}$ ；
- l ：barrier 同步时间。

这个公式非常重要。

它的含义是：

一个 superstep 的总时间由最慢的本地计算、最重的通信和全局同步开销共同决定。

本地计算越不均衡， $\max(w_i)$ 越大；
通信越拥挤， $\max(h_i)g$ 越大；
同步越频繁， l 的累计代价越大。

BSP 的优点：

- 把计算和通信分开；
- 模型比 PRAM 更真实；
- 程序结构清晰；
- 有利于分析性能；
- 有利于设计可移植并行程序。

缺点：

- 需要显式 barrier；
- 有些真实并行机不一定支持高效全局同步；
- 每个 superstep 里最多只能传 h 条消息，模型对通信有约束。

一句话概括：

BSP 是“本地异步 + 全局同步”的结构化并行模型。

4. More About BSP

第 4 页进一步比较 BSP 与其他模型。

4.1 BSP vs PRAM

课件给出结论：

```
If g = l = 1, then BSP = PRAM
```

意思是：
如果通信代价和同步代价都理想化为 1，那么 BSP 就退化成类似 PRAM 的模型。
这说明 PRAM 可以看成 BSP 的理想特例。

4.2 BSP vs Data Parallel

BSP 的 SPMD 编程模式很像数据并行语言。
SPMD 是：

```
Single Program Multiple Data
```

即多个处理器运行同一个程序，但处理不同数据。
这和数据并行的思想很接近，只不过 BSP 更强调 superstep 和通信同步。

4.3 BSP vs PVM/MPI

PVM/MPI 是实际的消息传递编程环境。
它们通常基于 pairwise synchronization，即点对点同步。
但它们缺乏一种简单的全局性能预测模型。
BSP 的优势在于：

- 有结构化的 superstep；
- 更容易判断程序正确性；
- 有性能预测模型；
- 在很多机器上实现更高效。

所以课件认为：
BSP 是比 PVM/MPI 更高层、更结构化的抽象。

4.4 BSP Library

课件列举了 BSP 编程库，例如：

- Green BSP library
- BSPlib
- Oxford BSPlibrary
- GPL
- BSP-L
- Opal

这些库提供类似：

- Initialization
- Enquiry
- Synchronization
- DRMA: Direct Remote Memory Access
- BSMP: Bulk-Synchronous Message Passing
- Halt
- Timer

也就是说，BSP 不只是理论模型，也有对应的编程库支持。

5. LogP Model

LogP 是比 BSP 更接近底层通信细节的模型。
它用于大粒度并行计算，适合分布式内存系统，也能用于共享变量、消息传递和数据并行范式。
LogP 名字来自四个参数：

参数	含义
L	Latency, 网络延迟
o	overhead, 通信开销
g	gap, 连续发送或接收消息之间的最小间隔

参数	含义
P	processors, 处理器数量

其中：

- **L** 表示消息从源到目的地的最大延迟；
- **o** 表示处理器参与一次发送或接收所花的开销；
- **g** 表示带宽限制，反映一个处理器连续发送消息的能力；
- **P** 表示处理器个数。

LogP 的特点是：

异步操作 + 消息通信 + 隐式同步

课件第 6 页说，在一个 superstep 中，处理器异步执行本地计算，并且最多发送/接收 $\lfloor L/g \rfloor$ 条消息。

LogP 的优点：

- 更能捕捉真实并行计算机的通信瓶颈；
- 能隐藏网络拓扑、路由算法和协议细节；
- 可用于不同并行编程范式。

缺点：

- 限制网络容量；
- 忽略通信竞争；
- 算法描述和设计比较困难。

可以这样理解：

BSP 适合结构化并行算法设计，LogP 更适合精细分析通信代价。

6. BSP vs LogP

第 7 页专门比较 BSP 和 LogP。

课件给出几个重要结论：

6.1 BSP 到 LogP 的关系

BSP 原本是：

Bulk Synchronization

如果把它改成：

Subset Synchronization

也就是不要求所有处理器一起同步，而只让部分处理器同步，就更接近 LogP。

6.2 BSP 和 LogP 可以相互模拟

课件说：

- BSP can simulate LogP with constant slowdown；
- LogP can simulate BSP with at most logarithmic slowdown。

意思是两者表达能力接近，只是性能模拟代价不同。

6.3 关系式

课件给出：

$$BSP = LogP + Barriers - Overhead$$

也就是说，BSP 可以看作在 LogP 基础上加入 barrier 抽象，同时隐藏一部分底层开销。

###6.4 总结

BSP 更适合算法设计和程序结构化；

LogP 更适合控制机器资源和分析底层通信。

课件结论是：

- BSP 更简单、可移植；

- LogP 更精细，但算法正确性和复杂度分析更麻烦。

7. C³ Model

C³ 是：

Computation, Communication and Congestion Model

即 **计算、通信与拥塞模型**。

它是分布式内存多计算机模型，适合粗粒度并行计算。

C³ 的主要特点是强调 **network congestion**，即网络拥塞。

C³ 把通信行为抽象为三个参数：

参数	含义
l	packet length, 包长度
s	start time, 启动时间
h	communication hops, 通信跳数

并且引入：

参数	含义
C_l	link congestion, 链路拥塞
C_p	processor congestion, 处理器拥塞

C³ 的计算仍由 superstep 组成，superstep 之间用 barrier 分隔。
在一个 superstep 中，处理器异步进行本地计算，并发送点到点消息。

C³ 的优点：

- 更详细考虑通信方案；
- 能反映 one-to-one、one-to-many 等通信模式；
- 能体现拥塞对性能的影响。

缺点：

- 参数更复杂；
- 需要设计者熟悉底层计算机结构；
- 算法设计和分析难度更大。

一句话：

C³ 比 LogP 更关注网络链路和处理器拥塞，是更底层、更复杂的性能模型。

8. 真实并行计算模型对比

第 9 页用表格比较 PRAM、APRAM、BSP、LogP 和 C³。

可以整理为：

模型	结构	计算	同步	参数	粒度	通信
PRAM	SIMD-SM	同步	自动	1	细/中	读写共享变量
APRAM	MIMD-SM	异步	Barrier	d, B	中/大	读写共享变量
BSP	MIMD-DM	异步	Barrier	p, g, l	中/大	发送/接收消息
LogP	MIMD-DM	异步	隐式	l, o, g, p	中/大	发送/接收消息
C ³	MIMD-DM	异步	Barrier	l, s, h	粗粒度	发送/接收消息

其中：

- SM: shared memory;
- DM: distributed memory;
- F/M: fine/medium grain;
- M/L: medium/large grain。

从抽象程度看：

PRAM 最抽象、最理想化
APRAM 稍微现实
BSP 结构化且实用
LogP 更贴近通信细节
C³ 更进一步考虑拥塞

9. 并行算法设计方法

接下来进入 **Design Methods of Parallel Algorithms**。

课件讲了三种大方法。

9.1 Parallelizing A Sequential Algorithm

第一种方法是：

在已有顺序算法中发现并利用并行性。

也就是把原来的串行程序改造成并行程序。

例如：

```
for i = 1 to n:  
    c[i] = a[i] + b[i]
```

这个循环中每次迭代互不依赖，就可以直接并行化。

课件指出：

优点：

- 最常用；
- 对很多普通程序有效；
- 尤其适合很多数值计算算法。

但也有问题：

- 不是所有顺序算法都能并行化；
- 最好的顺序算法不一定能变成好的并行算法；
- 如果顺序算法内部依赖很强，并行化效果会很差。

重点理解：

顺序最优 ≠ 并行最优。

有些算法串行很好，但并行性差；有些算法串行不是最优，但并行结构很好，反而更适合并行计算。

9.2 Designing A New Parallel Algorithm

第二种方法是：

根据问题本身重新设计并行算法，不依赖已有顺序算法。

这种方法通常能得到更高效的并行算法，因为它直接挖掘问题内在的并行性。

例如课件提到：

根据字符串周期性，在 SIMD-SM 模型上设计并行字符串匹配算法。

这种方法的特点：

- 创造性强；
- 难度高；
- 要求设计者有扎实的数学、算法和体系结构背景；
- 可能得到非常好的并行算法。

这是并行算法设计中更高级的一类方法。

9.3 Borrowing Other Well-known Algorithms

第三种方法是：

找出当前问题和某个经典问题之间的关系，然后借用经典算法思想。

例如课件给出的例子：

用矩阵乘法算法来解决 all-pairs shortest path，全源最短路径问题。

这是因为 Floyd-Warshall、最短路径闭包和矩阵乘法之间存在代数结构上的联系。

这种方法要求：

- 了解大量经典算法；
- 能发现问题之间的结构相似性；

- 有丰富的并行算法设计经验。

可以理解为：

把一个问题转化或类比到另一个已知问题，然后复用已有并行算法框架。

10. 并行算法设计技巧

课件强调，这些技巧不是直接套用的设计手册，而是一般性的指导思想。

10.1 Balanced Trees

Balanced Trees，即平衡树方法。

基本思想：

把输入元素构成一棵平衡二叉树；
从叶子向根进行归约；
再从根向叶子传播信息。

常用于：

- prefix-sum，前缀和；
- maximum/minimum，最大值/最小值；
- reduction，归约计算。

例如求 8 个数最大值：

第 1 层：两两比较
第 2 层：4 个结果两两比较
第 3 层：2 个结果比较
最后得到最大值

时间复杂度：

$t(n) = O(\log n)$

处理器数量：

$p(n) = n/2$

为什么是 $O(\log n)$ ？

因为二叉树高度是 $\log n$ ，每一层可以并行执行。

这是并行算法中最基础、最常见的技巧。

10.2 Doubling Technique

Doubling Technique 也叫：

- pointer jumping；
- path doubling。

核心思想是：

每一步把计算距离翻倍。

课件说：

经过 k 步后，每个元素已经获得距离 2^k 范围内的信息。

典型过程：

第 0 步：知道距离 1 的信息
第 1 步：知道距离 2 的信息
第 2 步：知道距离 4 的信息
第 3 步：知道距离 8 的信息
...

因此只需要 $O(\log n)$ 步就能覆盖整个链表或数组。

常见应用：

- list ranking，链表排名；
- finding roots of a forest，找森林中每个节点的根；
- 并行前缀类问题。

复杂度：

```
t(n) = O(log n)
p(n) = O(n)
```

直观例子：

如果每个节点有一个 parent 指针，pointer jumping 每次执行：

```
parent[x] = parent[parent[x]]
```

这样节点到根的距离每次大约减半，最终快速跳到根。

10.3 Partitioning Strategy

Partitioning Strategy 是划分策略。

基本思想：

把问题划分成若干个不重叠、大小近似相等的子问题；
然后并行求解这些子问题。

常见划分方法包括：

- uniform partition, 均匀划分；
- square root partition, 平方根划分；
- logarithmic partition, 对数划分；
- functional partition, 功能划分。

课件特别提醒：

Partitioning 不同于 Divide-and-Conquer。

Partitioning 的重点是：

一开始就小心划分，使得子问题结果容易合并。

它常用于并行排序、并行归并等问题。

10.4 Divide-and-Conquer

Divide-and-Conquer 是分治。

基本步骤：

1. **Divide**: 把原问题分成若干子问题；
2. **Conquer**: 递归求解子问题；
3. **Combine**: 把子问题解合并成原问题解。

它在顺序算法中非常有效，也自然适合并行化。
因为多个子问题可以同时求解。

课件给出的例子：

- bitonic merging network, 双调归并网络；
- convex hull, 凸包；
- DFT / FFT 类型分治计算。

10.5 Partitioning vs Divide-and-Conquer

第 17 页专门说明两者区别。

它们共同点：

都把问题分解成可以并行求解的子问题。

区别：

方法	重点
Divide-and-Conquer	主要工作在合并子问题解
Partitioning	主要工作在精心划分，使结果容易组合

换句话说：

- 分治：先拆，递归求解，最后合并可能很复杂；
- 划分：一开始就尽量拆得好，让后续合并简单。

10.6 Pipelining

Pipelining 是流水线技术。

基本思想：

把算法分成一系列阶段；
每个阶段的输出是下一个阶段的输入；
所有阶段以相同速率产生结果。

例如工业流水线：

阶段1：取指
阶段2：译码
阶段3：执行
阶段4：写回

在并行算法中也是类似的。

课件说：

- Pipelining 是并行处理中非常重要的技术；
- Systolic algorithm 是一种特殊的流水线算法；
- 例子是用 Horner 规则计算 4-point DFT。

流水线的关键条件是：

所有阶段必须以相同速率产生结果。

如果某个阶段特别慢，就会成为瓶颈，整个流水线速度由最慢阶段决定。

11. PCAM Design Methodology

PCAM 是并行算法设计中非常经典的方法论。

PCAM 代表四个阶段：

P: Partitioning
C: Communication
A: Agglomeration
M: Mapping

即：

划分 → 通信 → 聚合 → 映射

课件第 19 页图中展示了这个过程：

先把问题分成很多小任务，再分析通信关系，然后把小任务合并成更大的任务，最后映射到处理器上。

前两个阶段关注：

并发性和可扩展性

后两个阶段关注：

效率和性能

12. PCAM: Partitioning

Partitioning 的目标是：

■ 暴露并行执行机会。

课件用了一个很形象的比喻：

细沙比一堆砖更容易倒出来。

意思是：
细粒度分解能提供更多调度灵活性。

划分方法一般先看数据：

12.1 Domain Decomposition

领域分解，也叫数据分解。

先考虑数据结构，把大数据划分成多个区域。

例如二维网格计算中，可以把矩阵划分成多个小块，每个处理器负责一块。

12.2 Functional Decomposition

功能分解。

从操作或功能角度划分任务。

例如：

任务1：读取数据
任务2：滤波
任务3：特征提取
任务4：输出结果

课件建议：

优先关注最大的数据结构，或者访问最频繁的数据结构。

Partitioning 检查清单包括：

- 任务数是否比处理器数大一个数量级？
- 任务大小是否接近？
- 任务数是否随问题规模增长？
- 是否避免了冗余计算和冗余存储？

这些问题的目的都是为了保证：

并行性足够；
负载容易均衡；
算法能随问题规模扩展。

13. PCAM: Communication

Communication 的目标是：

交换任务执行所需的数据；
协调任务执行。

通信分析与划分方式有关。

###13.1 Domain Decomposition 中的通信

数据划分后，通信需求可能比较难判断。

因为你只是把数据分成了不相交的块，但计算操作可能需要访问邻居数据。

例如五点差分 stencil：

中心点需要上、下、左、右邻居的数据。

所以每个区域边界都需要和邻居通信。

13.2 Functional Decomposition 中的通信

功能划分中，通信一般更直接。
因为通信关系通常就是任务之间的数据流。

例如：

读取 → 预处理 → 计算 → 输出

数据从前一个功能模块传到后一个功能模块。

13.3 通信分类

课件列出四类通信分类：

分类	例子
local / global	五点差分是局部通信；集中求和是全局通信
structured / unstructured	mesh 是结构化；任意图是不结构化
static / dynamic	通信伙伴固定/变化
synchronous / asynchronous	协调式/非协调式通信

Communication 检查清单包括：

- 所有任务通信次数是否差不多？
- 每个任务是否只和少量邻居通信？
- 通信能否并发进行？
- 不同任务的计算能否并发进行？

重点是：

好的通信设计要减少通信量、避免热点、提高并发通信能力。

14. PCAM: Agglomeration

Agglomeration 是任务聚合。

目标是：

把小任务合并成较大任务，减少通信和软件开发成本，同时提高性能，并保持一定灵活性。

为什么要聚合？

因为 partitioning 阶段通常会故意产生很多细粒度任务。
但任务太细会带来：

- 通信过多；
- 同步过多；
- 调度成本高；
- 程序开发复杂。

所以需要适当合并。

课件提出一个重要概念：

Surface-to-Volume Effects

通信量通常与子区域的 **surface**，**表面积** 成正比；
计算量通常与子区域的 **volume**，**体积** 成正比。

当任务块变大时：

通信 / 计算 比例下降

例如二维区域中：

- 计算量约等于面积；
- 通信量约等于边界长度。

区域越大，边界/面积比例越小，因此通信开销相对减少。

Agglomeration 检查清单包括：

- 是否通过提高局部性降低通信成本？
- 聚合后的任务计算和通信成本是否相似？
- 任务数是否仍随问题规模增长？
- 是否牺牲了太多并发性？
- 是否导致负载不均衡、软件成本上升或可扩展性下降？

关键权衡是：

任务太细：通信和调度开销大；
任务太粗：并发性不足，负载不均衡。

15. PCAM: Mapping

Mapping 是映射。

目标是：

指定每个任务在哪个处理器上执行。

具体目标包括：

- 最小化总执行时间；
- 最大化处理器利用率；
- 最小化通信时间。

Mapping 的策略：

能并发执行的任务放在不同处理器上；
通信频繁的任务尽量放在同一处理器上；
使用负载均衡和任务调度提高映射质量。

课件指出，mapping 问题是 **NP-complete**。

这意味着：

一般情况下很难找到全局最优映射，只能使用启发式方法。

常见情况：

- 如果基于 domain decomposition，可以用 load balancing；
- 如果基于 functional decomposition，可以用 task scheduling。

Mapping 检查清单包括：

- 如果使用集中式负载均衡，管理者会不会成为瓶颈？
- 如果使用动态负载均衡，不同策略成本是否评估过？
- 如果使用概率或循环分配，任务数是否足够多以保证负载均衡？

Mapping 的本质是：

并发性、局部性、负载均衡之间的折中。

16. Switching Techniques

接下来进入 **Routing Techniques**，路由技术。

第 24 页介绍两种交换方式：

- Store-and-Forward Routing
- Cut-Through Routing

16.1 Store-and-Forward Routing

Store-and-Forward，存储转发。

基本过程：

每个中间节点必须先接收并存储完整消息；
然后再转发给下一个节点。

如果消息长度为 m ，路径长度为 l ，那么每经过一跳都要完整传一遍消息。

课件给出通信时间：

$$t_{\text{comm}}^{\text{SF}} = t_s + (m t_w + t_h) l$$

数量级：

$$O(m \cdot l)$$

其中：

- t_s ：启动时间；
- m ：消息长度；
- t_w ：每个字传输时间；
- t_h ：每跳开销；
- l ：路径跳数。

特点：

- 实现简单；
- 但长消息、多跳路径时开销很大。

16.2 Cut-Through Routing

Cut-Through, 直通路由。

基本过程：

消息头到达中间节点后，不必等完整消息到齐；
可以立即向下一跳转发。

它类似流水线传输。

课件给出通信时间：

$$t_{comm}^{CT} = t_s + m \cdot t_w + l \cdot t_h$$

数量级：

$$O(m + l)$$

相比 Store-and-Forward 的 $O(m \cdot l)$ ，Cut-Through 更快，尤其在消息长、跳数多时优势明显。

简单比较：

路由方式	是否存完整消息	时间复杂度
Store-and-Forward	是	$O(m \cdot l)$
Cut-Through	否	$O(m + l)$

17. X-Y Routing Algorithm

X-Y Routing 是二维 mesh 或 torus 中常见的维序路由算法。

课件第 25 页定义：

消息先沿 X 方向移动，直到到达目标处理器所在列；然后沿 Y 方向移动，直到到达目标处理器。

也就是：

先横向，再纵向

例如从 (2,1) 到 (7,6)：

$$\begin{aligned} (2,1) &\rightarrow (3,1) \rightarrow (4,1) \rightarrow (5,1) \rightarrow (6,1) \rightarrow (7,1) \\ &\rightarrow (7,2) \rightarrow (7,3) \rightarrow (7,4) \rightarrow (7,5) \rightarrow (7,6) \end{aligned}$$

优点：

- 简单；
- 路径确定；
- 容易实现；
- 可避免某些死锁。

缺点：

- 路径不一定负载均衡；
- 某些行或列可能形成热点；
- 不够自适应。

X-Y routing 是典型的 deterministic routing，确定性路由。

18. E-Cube Routing Algorithm

E-Cube Routing 常用于 hypercube，超立方体网络。

超立方体中，每个节点用二进制编号。

两个节点之间如果二进制编号只有一位不同，它们就直接相连。

E-Cube 的思想是：

比较 **source** 和 **destination** 的二进制地址；
找出不同的位；
每次沿一个不同维度翻转一位；
直到到达目标。

课件例子：

```
source S = 0110
destination D = 1101
```

路径之一：

0110 → 0111 → 0101 → 1101

每一步只改变一个 bit，对应超立方体中的一个维度。

E-Cube Routing 的特点：

- 简单；
- 适合超立方体；
- 路径长度等于源和目的地址的 Hamming distance；
- 是确定性维序路由的一种。

19. Cannon's Algorithm

接下来进入 **Dense Matrix Algorithms**。

Cannon 算法是并行矩阵乘法算法，适合在二维 mesh 上执行。

目标是计算：

$C = A \times B$

假设：

- A 和 B 是 $n \times n$ 矩阵；
- 处理器排列成 $p \times p$ 网格；
- 每个处理器负责一个矩阵子块。

课件描述的算法大致是：

19.1 初始对齐

对 A 的子块做循环左移：

第 i 行的 A 子块左移 i 步

对 B 的子块做循环上移：

第 j 列的 B 子块上移 j 步

这样处理器 $P(i, j)$ 上的 A 和 B 子块会对齐，能够开始乘加。

19.2 循环计算

每一轮：

1. 每个处理器计算本地 $A_block \times B_block$ ，并累加到 C_block ；
2. A 子块左移一步；
3. B 子块上移一步；
4. 重复 p 次。

因为每个处理器最终要累加 p 个子块乘积，所以执行 p 轮。

第 28 页图展示了一个 4×4 处理器网格上的 Cannon 移位过程：

A 块按行循环左移，B 块按列循环上移，经过多轮后每个处理器获得完整的 C 子块。

Cannon 算法的优点：

- 通信规则简单；
- 每轮只和邻居通信；

- 适合 mesh/torus;
- 负载均衡好。

缺点:

- 需要规则的二维处理器网格;
- 对矩阵维度和处理器数量有一定要求;
- 初始对齐有额外通信成本。

20. DNS Algorithm

DNS 是另一种并行矩阵乘法算法，使用三维处理器阵列。

课件第 29 页说：

n^3 processors are arranged as $n \times n \times n$ logical array.

也就是使用 n^3 个处理器，排列成三维逻辑数组。

每个处理器 $P(i, j, k)$ 有三个寄存器：

A, B, C

目标仍然是计算：

$C = A \times B$

矩阵乘法公式是：

$c_{jk} = \sum_i a_{ji} \cdot b_{ik}$

DNS 的核心思想是：

把所有乘积 $a_{ji} \cdot b_{ik}$ 并行算出来；
然后沿 i 维度求和。

算法步骤：

1. 把 A 和 B 的元素分布到 n^3 个处理器上；
2. 每个处理器计算一个乘积：

$C(i, j, k) = A(i, j, k) \times B(i, j, k)$

1. 对所有 i 方向上的结果求和：

$\sum_i C(i, j, k)$

得到最终的 $C(j, k)$ 。

第 30 页图用 $A_{4 \times 4} \times B_{4 \times 4}$ 展示 DNS 过程。

每一层 k 表示三维阵列中的一个平面，A 和 B 被复制或分布到不同维度上，然后每个位置计算局部乘积，最后沿某个维度归约。

DNS 的特点：

项目	说明
并行度	极高
处理器数量	n^3 ，非常多
计算时间	很低
通信/存储	开销大
适用场景	理论分析或超高并行度环境

可以理解为：

DNS 用大量处理器换取极高并行速度。

21. Gauss-Seidel Algorithm

最后讲解线性方程组求解中的 Gauss-Seidel 算法。

目标是求解：

$$Ax = b$$

设：

$$A = D + L + U$$

其中：

- D: 对角矩阵；
- L: 下三角部分；
- U: 上三角部分。

于是：

$$Dx = b - Lx - Ux$$

Gauss-Seidel 的核心思想是：

更新每个 x_i 时，尽量使用最新已经计算出来的值。

这通常比 Jacobi 方法收敛更快。

21.1 稠密矩阵情况下的并行化困难

课件指出：

如果 A 是 dense matrix，稠密矩阵，那么 Gauss-Seidel 有很强的顺序依赖。

因为：

$$x_i^{(k)} \text{ 必须先算出来，才能用于 } x_{i+1}^{(k)}$$

也就是说，同一轮迭代中，后面的变量依赖前面刚更新的变量。

这导致并行化困难。

为了并行化，通常需要改用 Jacobi 迭代公式。

Jacobi 的特点是：

$$\text{本轮所有 } x_i^{(k)} \text{ 都只依赖上一轮 } x_j^{(k-1)}$$

所以每个分量可以并行更新。

但代价是：

$$\text{Jacobi 通常收敛速度比 Gauss-Seidel 慢。}$$

21.2 稀疏矩阵情况下的并行化

如果 A 是 sparse matrix，稀疏矩阵，Gauss-Seidel 更容易并行化。

课件给出 Poisson PDE 的有限差分近似作为例子。

二维网格上，一个点的更新依赖它的上下左右邻居：

$$\begin{aligned} x(i, j) \text{ 依赖:} \\ x(i-1, j), x(i+1, j), x(i, j-1), x(i, j+1) \end{aligned}$$

这就是典型的五点差分结构。

为了并行化，可以使用 **red-black coloring scheme**，红黑着色方案。

21.3 Red-Black Coloring

红黑着色的思想是把网格点染成两种颜色：

```
红 黑 红 黑
黑 红 黑 红
红 黑 红 黑
黑 红 黑 红
```

这样每个红点的邻居都是黑点，每个黑点的邻居都是红点。

于是可以：

```
第一步：所有红点同时更新；
第二步：所有黑点同时更新。
```

同色点之间互不直接依赖，因此可以并行执行。

课件第 32 页图展示的就是网格点着色结构。

这个方法非常重要，因为它说明：

Gauss-Seidel 虽然天然有顺序依赖，但在特定稀疏结构下，可以通过图着色转化出并行性。

22. 本章核心总结

这一章的主线可以这样理解：

22.1 计算模型从理想到现实

```
PRAM → APRAM → BSP → LogP → C³
```

越往左：

- 模型越简单；
- 越适合理论分析；
- 越不贴近真实机器。

越往右：

- 越考虑真实通信；
- 越适合性能建模；
- 算法设计越复杂。

22.2 并行算法设计要解决四个问题

1. 有多少并行性？
2. 任务之间怎么通信？
3. 任务粒度多大合适？
4. 任务放到哪些处理器上？

这正好对应 PCAM：

```
Partitioning: 发现并行性
Communication: 确定通信关系
Agglomeration: 调整任务粒度
Mapping: 分配到处理器
```

22.3 常见并行技巧

```
Balanced Tree: 适合归约、前缀和、最大最小值
Doubling: 适合链表、树根、指针跳跃
Partitioning: 适合规则数据划分
Divide-and-Conquer: 适合递归结构
Pipelining: 适合阶段化计算
```

22.4 路由技术影响通信性能

Store-and-Forward: $O(m \cdot l)$
Cut-Through: $O(m+1)$
X-Y Routing: 适合二维 mesh
E-Cube Routing: 适合 hypercube

通信不是免费的。
并行程序性能经常不是被计算限制，而是被通信、同步、拥塞和负载不均衡限制。

22.5 典型算法体现不同并行思想

算法	体现的思想
Cannon	mesh 上邻居通信、循环移位、块矩阵乘法
DNS	三维并行、用大量处理器换时间
Gauss-Seidel	顺序依赖与稀疏结构并行化、红黑着色

23. 复习重点

这章最值得掌握的是：

1. PRAM、APRAM、BSP、LogP、C³ 的区别
尤其是它们的存储模型、同步方式、通信方式和适用粒度。
2. BSP superstep 代价公式

$Cost = \max(w_i) + \max(h_i)g + l$

1. LogP 四个参数

L, o, g, p

1. PCAM 四阶段

Partitioning → Communication → Agglomeration → Mapping

1. Store-and-Forward 与 Cut-Through 的复杂度区别

SF: $O(m \cdot l)$
CT: $O(m+1)$

1. X-Y routing 和 E-Cube routing 的基本路径选择规则
2. Cannon 算法的循环移位思想
3. DNS 算法的三维处理器阵列思想
4. Gauss-Seidel 为什么难并行化，以及红黑着色如何解决部分稀疏问题

这一章整体是在讲：

如何用合适的计算模型描述并行程序，如何设计并行算法，如何评估通信与同步开销，以及如何把算法映射到实际并行体系结构上。